

MovieMatcher: Graph and Agent Empowered Steerable Group Movie Discovery

David Abboud
University of Toronto
Toronto, Canada

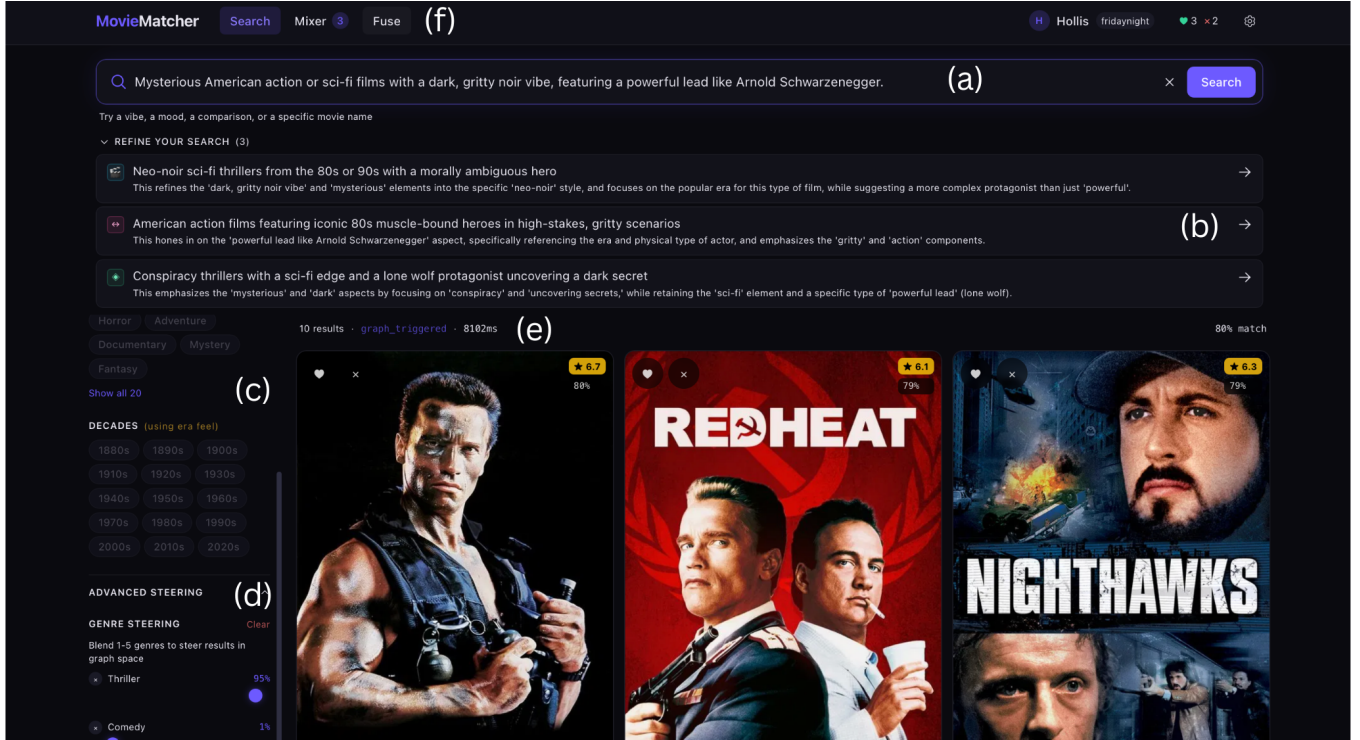


Figure 1: MovieMatcher allows collaborative movie discovery, where users browse individually, refine their own queries, and later enter a shared synthesis phase. Once the user signs in, they enter into the "Search" view. They can use the main search-bar (a) to express the movie they wish to watch, below which will appear query reformulation suggestions (b). One can filter and sort based on various metrics (c), or use the advanced genre and era steering features (d), which benefit from trained GAT embeddings of movies, genres, and era to refine the search. Results are displayed (e) after search is performed by finding relevant movies, given the filters. The top bar (f) allows switching to the Mixer view or Fuse view.

Abstract

Choosing what movie to watch is rarely a one-step decision. Instead, it is a process of exploring possibilities, refining vague preferences, and negotiating trade-offs—especially in group settings. We argue that movie recommendation should be approached as an interactive search problem rather than a static prediction task. We present MovieMatcher, a system that combines query reformulation, steerable controls, and graph-based exploration with collaborative agent-based synthesis. MovieMatcher allows users to iteratively shape their search through natural language, adjustable constraints, and example-based mixing, while providing AI-generated explanations to support interpretability. The system also introduces Fuse, a round-based interaction that summarizes shared preferences and disagreements and generates new search directions. This design shifts the role of recommendation systems from traditional systems

and chatbots to a framework where users and agents can collaborate together to reach consensus. We reflect on the challenges of balancing expressiveness and usability, and outline directions for designing exploratory and explainable retrieval systems.

CCS Concepts

• **Human-centered computing** → Interactive systems and tools; • **Information systems** → Recommender systems; • **Computing methodologies** → Knowledge representation and reasoning;

Keywords

interactive search, generative model, group recommendation, movie discovery, explainable recommendation, knowledge graph, query reformulation

1 Introduction

Choosing a movie is often framed as a lightweight recommendation task, yet in practice it is a richer interaction problem involving search, interpretation, and negotiation. Users seldom start with a fully specified request. Instead, they tend to articulate partial intent through mood, atmosphere, comparison, or soft constraints: wanting “something dark but not too heavy,” “a sci-fi movie with noir energy,” or “a movie like *Inception* and a bit like the MCU.” This ambiguity becomes even more pronounced in social settings, where the task is no longer to retrieve a title for a single individual, but to help several people explore possibilities, externalize their preferences, and converge on an option that feels enjoyable for everyone involved.

The specific context targeted in this work is the familiar but under-supported setting of group movie-night decision making. The MovieMatcher repository is built around exactly this use case: users join a shared session, explore movies individually through search and interactive controls, and later transition into a shared synthesis phase where the system helps the group surface overlap, disagreement, and next steps. This framing aligns with longstanding HCI perspectives on *exploratory search* and *mixed-initiative interaction*. Marchionini argues that exploratory search is not merely about locating a known item, but about learning, sense-making, and progressively understanding a search space [12]. Similarly, Horvitz’s account of mixed-initiative interfaces emphasizes that effective systems should not simply wait for perfect user input, but should actively assist users in articulating intent and navigating decisions [6]. MovieMatcher adopts these principles in the domain of movie discovery. It is designed not as a passive ranking engine, but as a partner in exploration: helping users reformulate their search, steer results, inspect why certain titles are being shown, and eventually reason about a group’s shared preferences.

Existing consumer systems only partially support this process. Some focus on media tracking and social sharing, such as Queue; others emphasize prompt-based recommendation, such as Maimovie [1, 2]. Conversational systems for streaming services similarly tend to emphasize chatbot-style recommendation flows, often centered on turn-by-turn retrieval or answer generation [4]. These systems are useful but provide less support for iterative reformulation, transparent refinement, and the explicit synthesis of multiple users’ search trajectories. In response, we present MovieMatcher, an interface that combines open-ended search, query reformulation, steering controls, graph-based exploration, and collaborative synthesis.

2 Background and Related Work

2.1 Exploratory Search and Mixed-Initiative Interaction

MovieMatcher is first grounded in HCI work. Marchionini notes that when users do not know exactly what they want, search becomes a process of learning, comparison, and refinement rather than simple lookup [12]. This perspective maps naturally onto movie discovery. Users often begin with a vague intention – a feeling, tone, or comparison – and need support in transforming that partial intention into a clearer search direction. This design stance

appears throughout our interface: search is accompanied by reformulations, every movie is shown with both AI summaries and an interactive neighborhood graph, thus helping users understand the space of possible options rather than merely retrieve a ranked list.

This orientation also aligns with foundational work on mixed-initiative interfaces. Horvitz argues that strong mixed-initiative systems should support a dynamic allocation of effort between user and system, with the system actively helping users make progress without taking away interpretive control [6]. MovieMatcher follows this model by treating user intent as provisional and revisable. Rather than assuming the initial query is final, the system proposes alternate phrasings, exposes refinement controls, and later uses other users preferences to shape AI summaries of movies as well as query reformulations. The design goal is not to automate movie choice away from the user, but to scaffold the process through which users articulate and negotiate what they want.

In addition, Gözükara et al. present a RAG-based interactive chatbot for video streaming services, demonstrating the appeal of conversational recommendation interfaces in this domain [4]. Their work confirms that users benefit from systems that can interpret natural-language requests and respond interactively. At the same time, MovieMatcher deliberately departs from the chatbot-only model. Rather than organizing the experience around sequential conversation alone, it adopts a multi-view interface in which search, reformulation, filtering, graph exploration, and collaborative synthesis coexist.

Among this year’s course papers, **DataScout** is the clearest direct influence on these choices. DataScout reframes dataset discovery as a task in which users struggle not only to retrieve relevant results, but also to formulate what they mean and to build a mental model of the search space [11]. Its use of AI-assisted reformulations, adaptive filtering, and dynamically generated relevance indicators provided an important interactional template for MovieMatcher. The influence is visible in the way reformulations are surfaced as actionable next-step queries rather than as mere writing advice, and in the broader decision to treat search as a guided exploration rather than a single prompt. In both systems, the interface is valuable not only because it returns results, but because it helps users progressively externalize and refine intent.

2.2 Steerable Interaction, Explanation, and Comparative Support

Another influence comes from HCI work on steerability, explanation, and the comparison of alternatives. **PromptPaint** is especially important here. Chung and Adar show that interaction with generative systems need not be confined to typing a better prompt; instead, users can benefit from more gradual and compositional forms of control [3]. MovieMatcher adapts this idea into the recommendation setting through genre steering, era steering, and movie mixing. Rather than requiring users to encode all nuance in text alone, the interface gives them controls for expressing preferences in terms of emphasis and proportion—for example, wanting more of one genre and less of another, or searching for something in between a set of reference films. In that sense, the system extends PromptPaint’s central lesson from image generation to exploratory retrieval.

As seen in class, Kim et al. show that explanations can increase user reliance, but reliance is not automatically appropriate simply because an explanation is present [9]. In MovieMatcher, generated summaries and justifications appear next to search results to help users decide whether a result seems aligned, whether they want to refine the query, or whether they want to continue exploring nearby options. Due to LLM hallucinations, the main search itself is not steered by the explanations, but rather the explanations are added post-hoc. Also, **GenAssist** demonstrates how AI-generated summaries of similarities and differences can help users compare multiple outputs without having to inspect each one in isolation [7]. This strongly influenced Fuse, where MovieMatcher synthesizes multiple users’ browsing histories into shared themes, points of disagreement, and a new group-level search direction. The paper I presented, **Generative Agents**, helped inspire the group-perspective AI reasoning, where the agent pretends to be different users and predicts whether they would like a movie or not. This is similar to the paper where AI systems can model multiple people as socially situated actors whose preferences and reactions evolve over time [14].

2.3 Knowledge Graphs, Retrieval-Augmented Recommendation, and the Movie Domain

A separate but equally important line of related work comes from recommendation research that combines knowledge graphs, retrieval, and language models. **KERAG_R** [13] pretrain item and entity representations over a bipartite item–entity graph, where movies are connected to entities such as genres, cast members, and other attributes. A graph attention network is used to compute attention over neighboring entities, and the resulting representations are trained with a contrastive hinge-style objective. This GAT is used to retrieve a small set of the most relevant knowledge graph triples for each candidate item, which are then passed to a fine-tuned language model. MovieMatcher builds on some of the same intuitions as KERAG_R; in our system, the graph is not only a source of retrieved evidence for text generation. Instead, it becomes part of the movie representation itself and directly supports the interaction layer through neighborhood browsing, genre steering, era steering, and graph-based retrieval.

Another important inspiration is **From Conversations to Knowledge: Enriching Movie Datasets with Knowledge Graph** [15]. It organizes movie information into a Neo4j knowledge graph with nodes such as movie, director, writer, actor, genre, and year, connected through explicit relations such as DIRECTED, WROTE, CAST_IN, and GENREOF. It also validates extracted triples against IMDb and Gemini, using IMDb as the stronger source of verification. This inspired our graph structure, the use of neo4j, as well as IMDB.

Finally, **Incorporating Domain Knowledge Graph into Multimodal Movie Genre Classification with Self-Supervised Attention and Contrastive Learning** is especially relevant because it shows how graph structure can shape the geometry of the learned movie space itself [10]. This work treats genres as meaningful anchors in the learned space through self-supervised and contrastive objectives, including centroid-based genre structure. This inspired MovieMatcher’s own use of graph-trained embeddings, as well as

the idea of shaping the space so that entities such as genres and decades are directions that meaningfully organize retrieval.

3 MovieMatcher: Interface

For a more clear description, please refer to the demo video at <https://youtu.be/RRKVAt-thnQ>. Also refer to the github repo at <https://github.com/DavidA00/MovieMatcher/>

3.1 Joining a party and session logistics

MovieMatcher begins with a shared party model. A user can either create a new party or join an existing one by entering a name and a party secret. Once inside, each user has their own search interface, but remains part of a common party context that tracks the current round and the other participants. The system gives users space to explore individually first, while still maintaining awareness of the shared group state.

3.2 Main search and Query Reformulation

The main search view begins with an open-ended search bar. Users can describe what they want in natural language, including mood, comparison, genre, tone, or other informal constraints. Even while the user is still typing a query, MovieMatcher dynamically presents personalized reformulation suggestions that reinterpret the user’s query from different directions, and we also show LLM reasoning for each. After a query is submitted, the interface returns a grid of movie candidates. Each result appears as a movie card with core metadata and a short AI-generated rationale. From there, the user can continue the search in several ways: they can like or dislike a movie, inspect it in more detail, apply filters, choose a reformulation, or switch into other views such as the mixer or Fuse. The search page therefore acts as the main hub of interaction.

3.3 AI summaries

Each movie result is accompanied by a short personalized AI-generated explanation that tells the user why the title may fit the current search, given their likes and dislike history and filters (see Figure 2(b)). From the user’s perspective, these summaries provide a quick rationale that can help the user decide whether to continue exploring a title, reject it, or revise the current query.

3.4 Interactive neighborhood graph

When a user opens a movie in detail, MovieMatcher presents not only the poster, metadata, and explanation, but also an interactive knowledge graph neighborhood. This view shows the local structure around the film through related genres, directors, actors, and other nearby concepts. Here, users can click on a director, and be prompted to search for movies with this director or ones with a similar style (likewise for cast as well as keywords) (see Figure 2(a)). Similarly, genre/ decade controls can be added from here.

3.5 Advanced filters: genre steering, era feel, and steerability

In addition to being able to apply hard genre and decade filters, set a minimum IMDb threshold, and sort by relevance, IMDb score, year, or title, users have access to advanced steering controls. Through

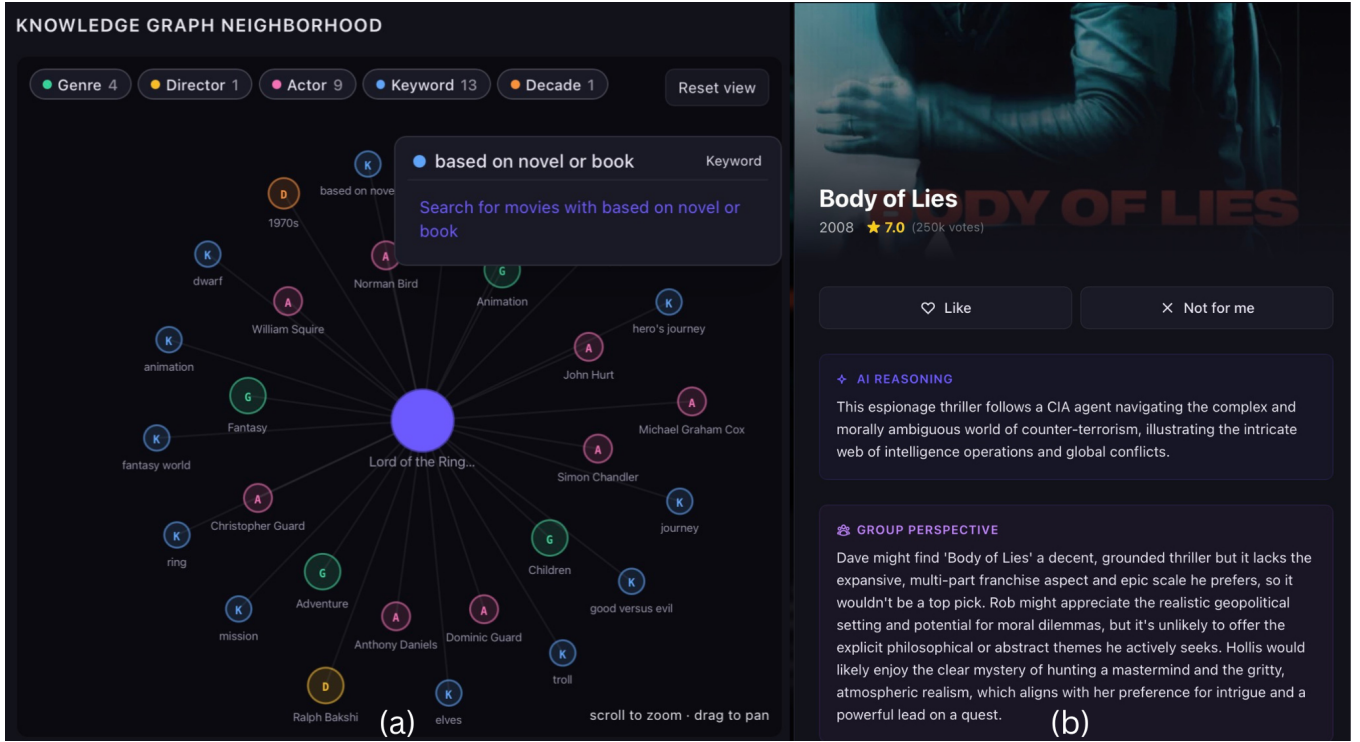


Figure 2: (a) Shows the interactive neighborhood graph (b) Displays the movie detail card, on which both the personalized AI summary and Group AI summary are seen.

Genre Steering, rather than simply toggling genres on or off, users can add several genres and adjust their relative weights. This allows the search to lean more toward some genres than others. A user can therefore express preferences such as wanting something mostly science fiction with some action, or horror with a smaller comedic component, without turning those categories into rigid constraints. Through Era Feel, users often want a movie that feels like the 1990s or early 2000s without necessarily wanting a strict decade filter. Era feel therefore acts as a softer stylistic preference. MovieMatcher also includes a steerability control, which adjusts how strongly the advanced steering should shape the returned results (See Filters in Figure 1).

3.6 Movie mixing

A user may want something halfway between two known films, or a group may want a movie that combines different people’s reference points. So, users can search for movies to add, adjust their weights, and look for films that lie at the intersection of those examples.

3.7 Fuse: collaborative synthesis

Fuse is MovieMatcher’s collaborative phase. Once users have browsed individually, they can switch to the Fuse view. After fusing, the interface presents a round summary, including similarities and differences. It crafts an AI search query for the group and executes it, returning candidate movies that will be liked by all. To add more

transparency, the LLM reasoning is shown here (See Figure 3). Starting in later rounds, MovieMatcher also introduces a stronger notion of shared context. The interface begins to surface group perspective reasoning for movies, where the agent predicts what each other user will think of that movie, given the last round’s fuse results. As a result, the interface changes over time. Early rounds emphasize individual exploration and broad search, while later rounds increasingly reflect the group’s accumulated preferences and earlier Fuse summaries (see figure 2(b)).

4 MovieMatcher: Technical Details

4.1 Implementation overview

The current codebase pairs a Next.js frontend with a FastAPI backend. Search and metadata are backed by Neo4j, while the LLM layer is implemented through a LangChain/LangGraph module in movie_agents.py, using Gemini models for reformulation, explanation, and group reasoning.

4.2 Data collection, structuring, and graph construction

The data is mainly collected from movielens, and merged with TMDB and IMDB [5, 8, 16], and is then formatted to remove missing items. The resulting graph contains node types for movies, genres, directors, actors, keywords, languages, countries, and decades. We then construct a rich textual representation for each movie before graph training. Each movie string concatenates title, year, decade,

overview, genres, directors, cast, keywords, language, and country. These texts are then encoded with `jina-embeddings-v5-text-nano` into 256-dimensional semantic vectors, which are later stored as `embedding_semantic`. At runtime, Neo4j stores both the graph structure and the learned vectors. We have two vector indices: `movie_semantic_idx` over `embedding_semantic`, and `movie_graph_idx` over `embedding_graph`. Search, steering, filtering, movie detail, and neighborhood lookup all query Neo4j directly.

4.3 GAT training and the learned movie space

The representation-learning stage is implemented in the notebook as a two-layer heterogeneous graph attention network, MovieGAT. All node types are first projected into a shared 256-dimensional space. The first GAT layer passes messages from entity nodes to movie nodes, allowing each movie to aggregate information from its neighboring genres, directors, actors, keywords, languages, countries, and decade. The second GAT layer passes messages from movie nodes back to entity nodes, so that the entity embeddings themselves become positioned meaningfully relative to the movies connected to them. Residual connections, layer normalization, dropout, and final L_2 -normalization are applied throughout. Four attention heads are used, with the standard GAT form. For a node i and one of its neighbors j , a learned compatibility score is computed as

$$e_{ij} = \text{LeakyReLU}(a^\top [Wh_i \parallel Wh_j]),$$

which is normalized across neighbors as

$$\alpha_{ij} = \text{softmax}_j(e_{ij}),$$

and then used to aggregate messages as

$$h'_i = \sum_{j \in N(i)} \alpha_{ij} Wh_j.$$

In MovieMatcher, this matters because not all neighbors should contribute equally. A movie such as *Alien*, for example, may attend differently to its horror, science-fiction, cast, and director neighbors depending on how those relations help organize it in the learned space.

The training objective is a weighted sum of three terms. The first is a knowledge-graph triplet loss over all entity types:

$$L_{kg} = \mathbb{E} \left[\max(0, \gamma + s(h_m, h_{e^-}) - s(h_m, h_{e^+})) \right],$$

where e^+ is a true neighboring entity and e^- is a random negative entity of the same type. In the code, $s(\cdot, \cdot)$ is implemented as a dot product between normalized embeddings, and the margin is set to 0.3. This term teaches that a movie should be closer to its real genres, actors, directors, and other neighbors than to unrelated ones. The second term is a semantic triplet loss over movie pairs mined from the semantic space:

$$L_{sem} = \mathbb{E} \left[\max(0, \gamma + s(h_i, h_k) - s(h_i, h_j)) \right],$$

where j is a semantically similar movie and k is a random negative movie. This term matters because two movies may be close in meaning even if they do not share cast or director nodes, and the semantic loss keeps that structure from collapsing during graph training.

The third family of terms are centroid losses for entity types that later become user-facing steering directions:

$$L_{centroid}^{(t)} = \frac{1}{|M|} \sum_{m \in M} \left\| h_m - \frac{1}{|N_t(m)|} \sum_{e \in N_t(m)} h_e \right\|_2^2.$$

This is applied specifically to genre, decade, and director, with different strengths:

$$L = \lambda_{kg} L_{kg} + \lambda_{sem} L_{sem} + \lambda_{genre} L_{genre} + \lambda_{decade} L_{decade} + \lambda_{director} L_{director},$$

where

$$\lambda_{kg} = 1.0, \quad \lambda_{sem} = 1.0, \quad \lambda_{genre} = 0.20, \quad \lambda_{decade} = 0.15, \quad \lambda_{director} = 0.05.$$

The centroid losses are what make genre, decade, and director entities usable later as steering directions in the interface, as inspired by [10].

4.4 How the main search works

The main search endpoint first encodes the user query into a semantic vector z_q , retrieves a larger pool of semantic candidates from `movie_semantic_idx`, and then averages the graph embeddings of the top n semantic hits to create a graph pivot:

$$g_q = \frac{\frac{1}{n} \sum_{i=1}^n g(m_i)}{\left\| \frac{1}{n} \sum_{i=1}^n g(m_i) \right\|_2 + \epsilon}.$$

It then retrieves graph-space neighbors from `movie_graph_idx` and combines the two scores as

$$\text{score}_{\text{hybrid}}(m | q) = \lambda \text{score}_{\text{sem}}(m | q) + (1 - \lambda) \text{score}_{\text{graph}}(m | q),$$

with $\lambda = 0.6$ by default.

In the live search endpoint, the system further expands the candidate pool adaptively. It starts with $k = 40$ candidates and can double this pool up to four times if post-filtering would otherwise leave too few results. Only after retrieval does the backend enrich metadata, apply optional preference re-ranking, apply IMDb-range filters, and sort according to the selected mode.

4.5 Query reformulation and AI summaries

The reformulate endpoint sends the current query to a prompt that asks for exactly three alternative searches along different dimensions such as mood, era, style, theme, or comparison. When a party has already completed a Fuse round, the prompt can also include group context so that some reformulations are subtly steered toward shared preferences. The `enrich` endpoint implements a second batched prompt that jointly generates reformulations, per-movie explanations, and filter suggestions in JSON form. The backend explicitly constructs a `results_block` from the top returned movies and adds user context from liked and disliked titles; if the user belongs to a party with prior Fuse rounds, the prompt also appends shared tastes and differences from the latest round summary.

4.6 Advanced filters: genre steering, era feel, and steerability

The search endpoint uses a composite function where the base vector, the weighted genre direction, and the averaged decade direction

are each normalized independently and then blended with explicit strengths:

$$v_{\text{comp}} = \text{norm}\left(\frac{w_q}{W} \hat{v}_q + \frac{w_g}{W} \hat{v}_g + \frac{w_d}{W} \hat{v}_d\right), \quad W = w_q + w_g + w_d.$$

This matters because it prevents a single large-magnitude component from overwhelming the others. In practice, the endpoint passes a `steering_strength` parameter that controls how strongly genre and era pulls should shape the final graph query.

4.7 Interactive neighborhood graph and graph-triggered search

The neighborhood graph shown in the detail view works by querying the selected movie’s one-hop outgoing relations. It then serializes these into a node-link structure suitable for frontend visualization. Clicking a node in this neighborhood can trigger a new search. The `graph_search` endpoint constructs a prompt containing the clicked node type and name, the source movie, the user’s likes, dislikes, and recent searches, and optionally group context from prior Fuse rounds. The LLM then produces a natural-language query, which is immediately run through the hybrid search pipeline.

4.8 Movie mixing

The mixer is implemented as a weighted centroid search over selected movies. When at least two mixer weights are provided, the backend fetches the selected movies’ graph embeddings, normalizes the weights, and computes

$$v_{\text{mix}} = \sum_i \tilde{w}_i g(m_i), \quad \tilde{w}_i = \frac{w_i}{\sum_j w_j}.$$

If advanced steering is active, this mixer vector is then passed through the same composite steering function described above; otherwise it is simply normalized and used as the graph ANN query.

4.9 Fuse, later rounds, and group influence

Fuse is implemented as a round-based group summarization and retrieval pipeline. The backend first constructs a textual profile for each party member from liked titles, disliked titles, and recent search queries. These are inserted into a prompt that asks for four outputs: similarities, differences, a rich group query, and reasoning. If the party already has earlier round summaries, the prompt also includes the previous similarities, differences, and group query so that the new round can build rather than restart.

The returned `group_query` is then fed directly into `hybrid_search`, and the resulting movie suggestions are filtered to remove titles that any member has already disliked. Starting in round 2 and beyond, group context influences other parts of the system as well. Both `enrich` and `reformulate` can include the latest Fuse summary so that explanations and reformulations are no longer conditioned only on an individual’s search state, but also on the group’s shared tastes and disagreements. In addition, `group_perspective` batches several movie candidates into a single prompt and asks for a short explanation of how each one would “land” with the group, naming individual members where possible.

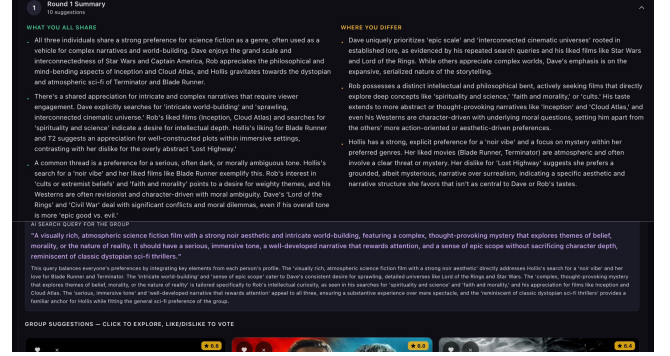


Figure 3: Overview of the Fuse Interface, showing the similarities, differences, and suggested query and movies.

5 User Study

We conducted a small formative user study with two participants who explored the system and provided feedback on the clarity of the interface and the intelligibility of its controls. The study led to five concrete revisions. First, the like and dislike controls on movie cards were made larger and visually clearer, so that preference feedback would feel more immediate and less error-prone. Second, the exposed parameter that let users tune the balance between graph and semantic search was removed from the interface and fixed to a default value as users found it confusing. Third, because participants found the filtering system difficult to interpret, the interface was reorganized. Fourth, genre steering was redesigned from a rigid two-genre interaction into a more flexible control that allows users to add or remove between one to five genres and adjust their relative strengths. Finally, more explanatory text was added throughout the interface to explain how the platform should be used. Although small in scale, this formative study reinforced a central HCI lesson that also appears in DataScout and PromptPaint: when a system offers novel forms of steering and interpretation, the design challenge is not only whether the feature exists, but whether users understand when to use it and what it will do [3, 11].

6 Limitations

MovieMatcher has several limitations, some of which are about data coverage, some about explanation and trust, and others about the broader interaction model the system currently supports. The first limitation is the scope of the movie corpus itself: it only covers movies up to 2024, and includes various less familiar or non-English titles. It is worthy to note that this can be seen as a strength as it points users to less popular titles, hence allowing the discovery of “hidden gems”. Furthermore, MovieMatcher presents AI-generated reasoning for why a title may fit a user or a group, but those explanations are generated after retrieval, which risks encouraging users to believe that the visible rationale is the true cause of a recommendation. Interestingly, this mirrors the problem with LLM think tokens in general, which have been proven to not necessarily reflect the true reasoning taken to reach a response. [9]. Finally, the current user study was intentionally formative and very small due to lack of time and resources, which made it useful for catching interface issues early, but insufficient for making

broader claims about usability, trust, recommendation quality, or collaborative decision-making outcomes.

7 Future Work and Potential Evaluation

One clear direction for future work is to strengthen the system's support for automatic filter inference. A promising next step would be a pre-search Agent that interprets a user's query together with the current state of the interface and suggests which hard and soft constraints should be activated automatically. The challenge here is that the system should make clear which parts of the user's input are being treated as hard constraints, which are being interpreted as soft preferences, and how those choices are reflected in the interface. However, one must be weary lest they fall into the danger of taking too much control away from the user. Another direction is to use generated AI explanations as a second-pass filtering mechanism: if a rationale suggests poor alignment or includes signals of mismatch, the corresponding result could be down-ranked or removed. This would move explanations from being purely descriptive to being operationally meaningful inside the search loop. Also, a similar system can be extended to music, series, and even other search settings such as rental discovery. The core interaction pattern underlying MovieMatcher is not inherently tied to film: users express partial intent, receive reformulations, shape results, inspect why candidates fit, and negotiate a final choice with others. That pattern could plausibly generalize to many domains where preference expression is incomplete and where shared decision making is important. Lastly, the system's support for group recommendation could be expanded. For example, the system could distinguish between options that maximize consensus, options that preserve minority tastes, and options that introduce compromise. It could also make the logic of compromise more visible by showing how different candidate titles satisfy different parts of the group's profile. If I had the chance to conduct a thorough study on a component of the system, I would choose the Fuse feature. I would run a user study randomly assigning participants into groups of 4, asking them to use my system VS a classic Chatbot to find results that the whole group might like. Then, they would have to rate on a Likert scale various aspects of each like diversity of results, difficulty of usage, ease of use, satisfaction with personal recommendations, with reasoning provided, and with end outcome. We could also study the effect of varying the group size, and participant movie expertise. Longer-term studies could also help reveal how users understand and use reformulations, whether graph exploration improves decision quality or only adds novelty, and how much explanations influence trust.

8 Conclusion

MovieMatcher is best understood not simply as a recommender, but as an HCI contribution that explores how retrieval, explanation, and collaboration can be brought together in a single exploratory interface. By reframing movie selection into a mixed-initiative search process, we provide users with the agency to iteratively refine their intent through natural language, structural graph exploration, and steerable controls. The introduction of the Fuse interface and the integration of GAT-based structural embeddings suggest a path

forward for design where transparency and steerability are prioritized over "black-box" recommendations. Ultimately, MovieMatcher offers a blueprint for the next generation of exploratory retrieval systems to empower users and agents to work together in harmony.

References

- [1] Maimovie: AI movie & TV finder. <https://apps.apple.com/ca/app/maimovie-ai-movie-tv-finder/id1488047888>. App Store. Accessed: 2026.
- [2] Queue: Where to watch movies, TV shows, trailers & reviews. <https://www.queue.co>. Accessed: 2026.
- [3] John Joon Young Chung and Eytan Adar. PromptPaint: Steering text-to-image generation through paint medium-like interactions. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023.
- [4] Hamza Gözükar, Jay Patel, Erkan Kara, Ayşenur Yıldız, Ozan Köseoglu, Didem Makaroglu, Yassine Drias, Şirin Özlem, and Tuna Çakar. RAG based interactive chatbot for video streaming services. In *Proceedings of the 10th International Conference on Computer Science and Engineering (UBMK)*, pages 1350–1355. IEEE, 2025.
- [5] F. Maxwell Harper and Joseph A. Konstan. The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems*, 5(4):19:1–19:19, 2015.
- [6] Eric Horvitz. Principles of mixed-initiative user interfaces. In *Proceedings of the ACM SIGCHI Conference on Human Factors in Computing Systems (CHI '99)*, pages 159–166. ACM, 1999.
- [7] Mina Huh, Yi-Hao Peng, and Amy Pavel. GenAssist: Making image generation accessible. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023.
- [8] IMDb. IMDb: Internet movie database. <https://www.imdb.com>, 2026. Accessed: 2026.
- [9] Sunnie S. Y. Kim, Jennifer Wortman Vaughan, Q. Vera Liao, Tania Lombrozo, and Olga Russakovsky. Fostering appropriate reliance on large language models: The role of explanations, sources, and inconsistencies. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, 2025.
- [10] Jiaqi Li, Guilin Qi, Chuanyi Zhang, Yongrui Chen, Yiming Tan, Chenlong Xia, and Ye Tian. Incorporating domain knowledge graph into multimodal movie genre classification with self-supervised attention and contrastive learning. In *Proceedings of the 31st ACM International Conference on Multimedia*, 2023.
- [11] Rachel Lin, Bhavya Chopra, Wenjing Lin, Shreya Shankar, Madelon Hulsebos, and Aditya G. Parameswaran. Rethinking dataset discovery with DataScout, 2025.
- [12] Gary Marchionini. Exploratory search: From finding to understanding. *Communications of the ACM*, 49(4):41–46, 2006.
- [13] Zeyuan Meng, Zixuan Yi, and Iadh Ounis. KERAG_R: Knowledge-enhanced retrieval-augmented generation for recommendation, 2025.
- [14] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pages 1–22, 2023.
- [15] Darshna Parmar, Pramit Mazumdar, Tanish Nagrani, and Rushabh Babre. From conversations to knowledge: Enriching movie datasets with knowledge graph. In *Proceedings of the 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, pages 1–7. IEEE, 2024.
- [16] The Movie Database. TMDb: The movie database. <https://www.themoviedb.org>, 2026. Accessed: 2026.